Scripts

- A file that contains multiple sequential lines of commands and function calls.
- Once a script is run (by typing its name in the command window and pressing enter), the commands are executed one after the other automatically.
- Unlike functions, scripts do not have input and output parameters (arguments or variables).
- All variables that are created in a script are added to the workspace.

Scripts

- E.g.
- A script to
  1. create a 2 x 2 matrix, A of elements 2, 3 on the first row and 0, 5 on the second row
  2. Create a 2 x 2 matrix B whose four elements are all 1
  3. Find the sum of matrices A and B and assigns it to matrix C

will be entered in the editor windows as;

```
%    This script creates two 2x2 matrices and finds their sum.
A = [2,3; 0,5]    % creates matrix A of elements 2,3 on row 1 and 0, 5 on row 2
B = ones(2,2)     % creates 2x2 matrix of ones
C = A + B
```

- Assuming the file is saved as "trial script",
  anytime the name trial script is run in the command window;

```
Command Window
>> trial_script
A =

    2    3
    0    5

B =

    1    1
    1    1

C =

    3    4
    1    6
```

Scripts

- A Semi colon at then end of a executes the command "silently", without displaying results in the command window. So;

```
%    This script creates two 2x2 matrices and finds their sum.
A = [2,3; 0,5]    % creates matrix A of elements 2,3 on row 1 and 0, 5 on row 2
B = ones(2,2)     % creates 2x2 matrix of ones
C = A + B
```

```
>> trial_script
C =

   3   4
   1   6

>>
```


Scripts: Some recommended practices

- When saving scripts, use names you can easily remember/identify with.
- Include a description of the script with comments. You can also add a description to each line of the script.
- Precede comments with the % symbol. Anything after the % symbol on a line is not executed.

User Interaction with Scripts and Functions

- It is possible get different results from a script without actually modifying the script.
- This can be done by entering data "interactively".
- Making scripts "interactive" also makes them more user-friendly.
- By assigning to the variable

```
variable = input('Enter variable name: ')
```

- The script pauses and will only continue after an input has been typed in.
- E.g.

  Modify Tutorial 2 script to interactively ask for the name of the excel (or text file) to load, and the elements of Matrix B.

Functions

- Expressions that accept one or more input values and calculates and displays desired outputs in a desired format.
- Often, functions in Matlab/Octave are defined for one or more scalar inputs and produce a scalar output. However, if functions receive an array of input values, then they will calculate an array of output values on an element-by-element basis.
- Unlike scripts, variables created in functions are not automatically stored in the workspace. They must be assigned to a variable in the command window first.
- A function is initialized by typing its name, and the required inputs (in the required order) in brackets.
- Functions can also be made interactive.
- Functions can be "called" in scripts.

User Defined Functions

```
function [output_arg1,... ] = function_name(input_arg1,... )
%    Summary of this function goes here
%    Detailed explanation goes here

end
```

E.g.

```
function [area, cf] = circle(r)
%   This function takes radius of a circle as input and calculates the area (ar) and
%   circumference (cf) for the said circle.
area = pi*r^2
cf = 2*pi*r
end
```


Functions

Function circle can be run (for a radius of 1 for instance) in the command window as

```
>>  circle(1);
```

To assign both outputs to variables, call the function as;

```
>> [area, cf] = circle(1)
```